Otto-von-Guericke-University Magdeburg
Faculty of Electrical Engineering and Information Technology
Institute for Automation Engineering

Master Thesis



Wireless Industry 4.0 Negotiation Scenario

Author:

Parva Yazdani
Matriculation No.: 224062
May 04, 2022

Supervisors:

Prof. Dr.-Ing. Christian Diedrich
Chair of Integrated Automation
Otto-von-Guericke-University Magdeburg
Universitätsplatz 2
39106 Magdeburg, Germany

Dr. Lutz Rauchhaupt
Senior Engineer
Institute for Automation und Communication
Werner-Heisenberg-Strasse 1
39106 Magdeburg, Germany

Abstract

In Industry 4.0 applications, communication systems should adapt themselves to the possible required changes. This is a pre-condition to have a smart production, which leads to a flexible production process. For this reason, the data exchange between the production system and the wireless system is necessary.
In this work, a digital representation of a use case negotiation scenario is developed in OMNeT++ using the concept of asset administration shell. This makes the data exchange among industrial assets and between assets and the wireless system possible. As a result, the wireless system will have access to the asset-related data continuously and will be able to keep the use of the wireless medium at an efficient level.
In the second part of the work, the basic module to simulate communication scenarios in SIMBA is generated, and a roadmap for porting our specific negotiation scenario from OMNeT++ to SIMBA is documented. This shows that it is possible in the furture to use SIMBA as a network simulator to model communication systems in a graphical way.

OTTO VON GUERICKE UNIVERSITÄT MAGDEBURG | EIT | FAKULTÄT FÜR ELEKTROTECHNIK UND INFORMATIONSTECHNIK

Masterarbeit

For Ms. Parva Yazdani
(Matrikelnr.: 224062)

Wireless Industry 4.0 negotiation scenario

Industry 4.0 applications place completely new demands on communication networks and services. While today's networks are firmly planned with regard to the production facilities, communication in the future must support flexible production processes. Wireless communication is a key component in these future communication architectures.
In order to support investigations and developments of Industrie 4.0 scenarios, basic components of a digital twin are to be developed. With this, a scenario to be specified is to be created and validated.

The required work includes following tasks:

1. Literature research on the asset administration shell (AAS), its use and development
2. Development of a concept for negotiations between application system and communication system according to Industrie 4.0
3. Development of a concept for the use of AAS with Omnet++ and/or Simba.
4. Exemplary implementation of AAS with relevance to Industrie 4.0 oriented negotiations
5. Development of a roadmap for porting Omnet models to Simba
6. Validation of an Industrie 4.0 negotiation using Omnet++ or Simba

Magdeburg, 9.11.2021

Date of issue:	01.12.2021
Date of submit:	04.05.2022
First Examiner:	Prof. Dr.-Ing. C. Diedrich
Second Examiner:	Dr.-Ing. L. Rauchhaupt
Supervisor:	Dr.-Ing. L. Rauchhaupt

Prof. Dr.-Ing. Christian Diedrich
Assignor of the task (First Examiner)

Prof. Dr.-Ing. Roberto Leidhold
Chair of the Examination

Declaration by the candidate

I hereby declare that this thesis is my own work and effort and that it has not been submitted anywhere for any award. Where other sources of information have been used, they have been marked.

The work has not been presented in the same or a similar form to any other testing authority and has not been made public.

I hereby also entitle a right of use (free of charge, not limited locally and for an indefinite period of time) that my thesis can be duplicated, saved and archived by the Otto von Guericke University of Magdeburg (OvGU) or any commissioned third party (e. g. *iParadigms Europe Limited*, provider of the plagiarism-detection service "Turnitin") exclusively in order to check it for plagiarism and to optimize the appraisal of results.

Magdeburg, April 18, 2022

Acknowledgements

I would like to thank my primary supervisor, Dr. Lutz Rauchhaupt for this great opportunity that while challenging has tought me a lot. His immense support, guidance and encouragement throughout this project paved the way for me. Also, I want to specially thank Gustavo Cainelli for his contribution in this thesis. Indeed, this work could not be done without his generous assistance and dedication.
Many thanks to Dr. Jens Alex for his help and support regarding SIMBA. I would also like to take this opportunity to thank Professor. Dr.-Ing. Christian Diedrich for his help and ideas regarding AASX Package Explorer.
Finally, I would like to express my gratitude and appreciation to the Institute for Automation and Communication (IFAK) for giving me the chance to learn and grow by doing my thesis at their innovative institute.

Contents

1 Introduction **5**

1.1 The evolution of industrial production 5

1.2 Motivation 7

1.3 Objectives 7

1.4 Digital twin 8

2 Related works **12**

2.1 Model railway demonstrator system 12

2.2 Other related works on AAS 13

3 Model description **15**

3.1 Wireless industrial automation 15

3.2 Industry 4.0 scenario for managing communication resources 17

3.3 Detailed behavior of each AAS 19

3.4 Definitions and mathematical calculations 24

3.5 The structure of each AAS 25

4 Communication models in SIMBA **34**

4.1 Motivation 34

4.2 TicToc example in SIMBA 34

4.3 Negotiation scenario in SIMBA 37

5 Results and Discussion **39**

6 Conclusions and Future work **41**

Bibliography **42**

List of Acronyms

I4.0 Industry 4.0

CPS Cyber physical system

WCS Wireless communication system

AAS Asset Administration Shell

MoRa Model Railway Demonstrator System

OPC-UA OPC Unified Architecture

AASX PE AASX Package Explorer

WSM Wireless system manager

TCP Transmission Control Protocol

UDP User Datagram Protocol

HTTP Hypertext transfer protocol

MQTT Message queuing telemetry transport

List of Figures

1.1 The evolution of industry 1.0 to 4.0 [2] 6
1.2 Digital twin [6] 8
1.3 The structure of AAS 9
1.4 Three types of asset administration shell interaction pattern [10] 10

2.1 Process flow of MoRa [20] 12
2.2 AAS related elements of the OI4 reference architecture [21] 13

3.1 Wireless industrial automation as a system of systems [27] 15
3.2 Visualization of the physical and digital domain in the concept of I4.0 . . . 16
3.3 System classes of the use case scenario 17
3.4 State diagram of the customer terminal 20
3.5 State diagram of the product 21
3.6 State diagram of the machine broker 21
3.7 State diagram of the machine 22
3.8 State diagram of the wireless system manager 23
3.9 The scenario for the wireless manager and the use of AAS 24
3.10 I4.0 language in AASX PE 26
3.11 Structure of customer terminal in AASX PE 27
3.12 Structure of product in AASX PE 29
3.13 Structure of machine broker in AASX PE 30
3.14 Structure of machine in AASX PE 32
3.15 Structure of wireless system manager in AASX PE 33

4.1 Parameter block window in SIMBA 35
4.2 Block configuration window in SIMBA 37
4.3 TicToc example in SIMBA 37
4.4 Use case scenario in SIMBA 37

5.1 Use case scenario in OMNeT++ 39

List of Tables

3.1 The product types and the corresponding required functions to produce them. 18
3.2 The functions and the corresponding capable machines to operate them. . 18
3.3 The functions and the corresponding transfer intervals. 18
3.4 The state-transition table of customer terminal 27
3.5 The state-transition table of product 28
3.6 The state-transition table of machine broker 30
3.7 The state-transition table of machine 31
3.8 The state-transition table of wireless system manager 33

1 Introduction

In this chapter, first a background about Industry 4.0 (I4.0), and its importance is given. This follows with the motivation and the objectives of this work. Finally, the basics of the taken approach are discussed.

1.1 The evolution of industrial production

For centuries, most of goods were manufactured by hand or work animals. In the late of 18th century, industrial evolution began. Figure 1.1 shows the progress from I1.0 to I4.0.

- I1.0 came with the mechanical manufacturing facilities, i.e. water and steam powered machines, to increase production quality, efficiency and scale.
- I2.0 came with machines running on electrical energy which were more efficient in cost and effort compared to water and steam based machines. I2.0 also came with mass production using assembly line.
- I3.0 came with automated machines which in some cases were even a complete replacement for the human agent. This created a requirement of software systems to control not only the machines, but also the whole manufacturing process. Programmable logic controller (PLC) was invented to do so. In this era, the entire industry was automated using electronics and IT.
- I4.0 came with Cyber physical systems (CPSs) to merge the boundaries of physical and virtual world by allowing machines communicate more intelligently with each other. CPSs make the machines smarter which results in lower downtime for industries. This also makes it possible to manage the industry completely remotely from another location. [1]

The emergence of using I4.0

Usually, the traditional manufacturing systems have mass production and, the number of products that will be produced is known. This kind of system is not flexible and is static, which means the production system is built to produce a certain kind of product and will produce it until the system is reconfigured, usually by a human. It is also well-engineered, which means that everything is planned and, in some cases, it is planned for the worst-case which leads to a non-efficient use of resources.

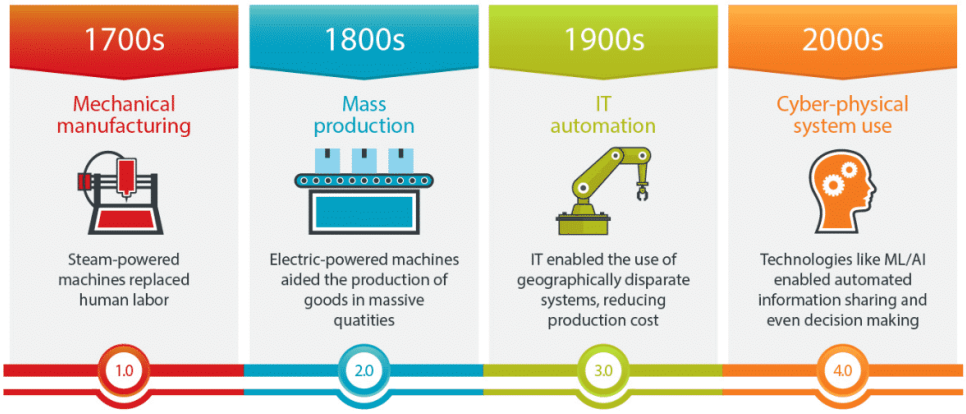


Figure 1.1: The evolution of industry 1.0 to 4.0 [2]

Considering the high demand for product customization, the modern world is concerned with the emergence and use of I4.0 to take a step towards flexible production. I4.0 came with new features to deliver maximum outputs with efficient resource utilization, enabling a variety of different products to be produced in the same production facility [3]. Some of these features:

- Flexibility, which means adapting the use of the production system to the needs of customers, without planning every detail of the production system in advance.
- Virtualization, which means that some parts of the production system will have virtual copies. It enables the remote acquisition of information, as well as the observation and control of the environment, to save time, reduce the amount of error and, make better decisions.
- Asynchronous manufacturing, meaning that the components in the production flow use auto-identification technology to inform each machine what needs to be done to produce customized products at each step of the production process.
- Self-configuring capabilities, which means that the system is able to configure itself without human intervention. The machine will not depend on the human element to be configured. It will be reconfigured dynamically by a digital system, which receives information on how the system is working and controls production in real-time.

Communication in I4.0

The communication system in industry is formed by wired and wireless systems. In I4.0 the wireless systems are gaining further momentum.
Wireless communication system (WCS) includes all forms of communication between two or more devices using wireless signal through wireless communication technologies and devices. It has several advantages over the wired communication system:

- By eliminating connection wires there would be no need for complicated physical infrastructure and maintenance efforts, therefore it comes with cost effectiveness and easy installation.
- It increases the flexibility and convenience since the location is not a matter anymore [4].
- It allows devices to move freely without getting disconnected. In other words, it increases the mobility.

Three main components of a typical WCS are transmitter (Source), channel and receiver (Destination). There exists different wireless communication technologies such as Wi-Fi, bluetooth, infrared, satellite, and mobile communication systems [5].
The fifth generation of mobile communication systems (5G) promises to cope with several industrial requirements. It is highly reliable, secure and helps to have a more flexible production. Therefore, there exists a high demand of this technology in industrial automation.

1.2 Motivation

Since WCS is a crucial part of highly interconnected I4.0 systems, it has to cope with the level of adaptability required by I4.0 and, should not be planned and installed for worst-case scenarios as in the traditional systems. It means that if changes are necessary, the WCS and/or its use of the medium should be adapted accordingly. It includes changes due to the production process requirements or changes due to external disturbances, for example, interferences. For that, WCS should be capable of self-configuration to adjust itself to the possible changes. This is a pre-condition to have a smart production which leads to a flexible production process. To reach this capability, a collaboration between the production system and WCS is necessary. To enable this collaboration between the parties, a digital twin approach is developed in this work. The digital twin and its implementation will be described in section 1.4.

1.3 Objectives

This work consists of two main parts:

1. The development of a concept of resource negotiation in I4.0 scenarios.
   The main objective of this part is to prove the concept of communication between the application system and the wireless system manager by implementing a use case scenario according to Industry 4.0 in OMNeT++ simulation software.
   The side objectives are:

- To take a step closer to the flexible production process.
- To show the importance of AAS by creating a systematic form of the assets of the use case scenario in AASX package explorer.

2. The modelling of message exchange in SIMBA.
   The objective of this part is to show the possibility of using SIMBA to model communication systems. It is important to highlight that, this simulation software was not originally programmed to do this.

1.4 Digital twin

The digital twin is a digital representation that reproduces the behavior of an entity. It is used to describe, simulate, predict the state and behavior of its real-world counterpart, based on the computational models and real-time data. The digital twin is capable to find the parameters that optimize the system before the installation and also, to respond optimally to changing conditions of the real-world twin during the operation phase. To access the data, invoke the commands, and run the models (See Figure 1.2) digital twin provides a service interface. In other words, this service interface will enable the communication between digital twin and software applications [6].

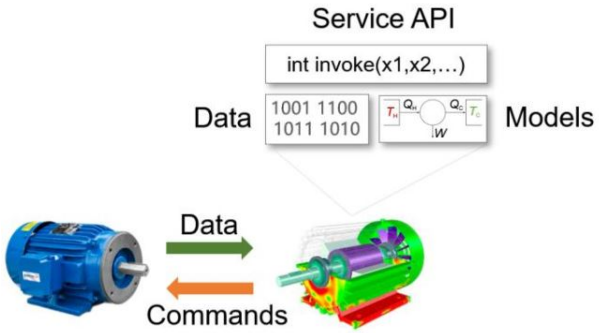


Figure 1.2: Digital twin [6]

Platform I4.0 proposes the Asset Administration Shell (AAS) as an approach to implement the digital twin [7].

Asset administration shell

According to Platform I4.0 interoperability is essential to have flexible production. AAS is the central concept of Platform I4.0 to facilitate interoperability among applications of a company and across companies by implementing the digital twin. In other words, it is a standardized digital representation of an entity here called an asset, which contains all information that represents the characteristics and behavior of that asset [6, 8]. According

to [8], asset is an entity owned by or under the custodial duties of an organization, which has either a perceived or actual value to the organization. They can be devices, machines, components, documents, or even software.

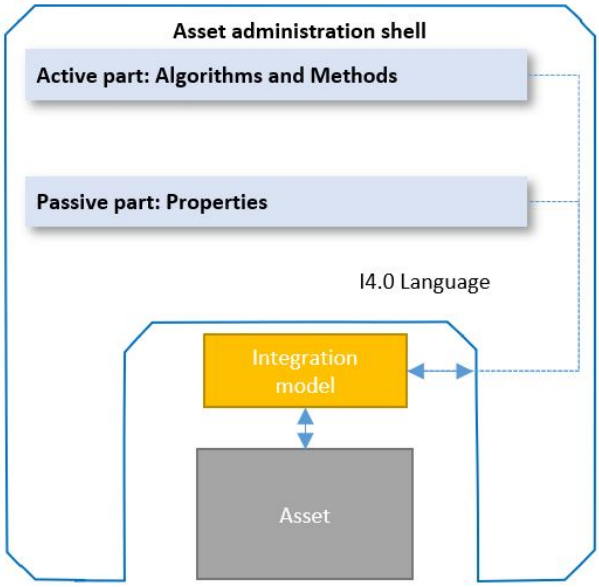


Figure 1.3: The structure of AAS

Through the AAS every asset becomes an I4.0 component. AAS organises and manages all the information from the asset in a systematic form. It means the asset will be surrounded with an AAS that is made up of a series of submodels in which, different aspects of that asset are stored [9].
As shown in Figure 1.3, these aspects have an active and a passive part. Active part contains the algoritm and methods, and passive part contains the properties that charactrerize an asset. Other AASs can also reach these submodels. Through the integration model, physical world transits into the digital environment by adapting to I4.0 language.
According to Platform I4.0 [7], there exists three types of interaction with the AAS with three different technical requirements. As is shown in Figure 1.4 in case of a passive pattern the AAS is a file package which contains the information of the asset. These information are written in a uniform data format e.g XML or JSON, and will be exchanged among value-chain partners. The second type is when information will be exchanged through web-based protocols e.g HTTP and MQTT. This is called reactive interaction. In the third pattern the AASs will interact using a standardized I4.0 language. This enables the peer-to-peer AAS communication.
In this work, an AASX file package format specified by Platform I4.0 as an standardized I4.0 language is created. This file encapsulates AAS information in a structural way. Therefore it can be indicated that proactive pattern has been used.

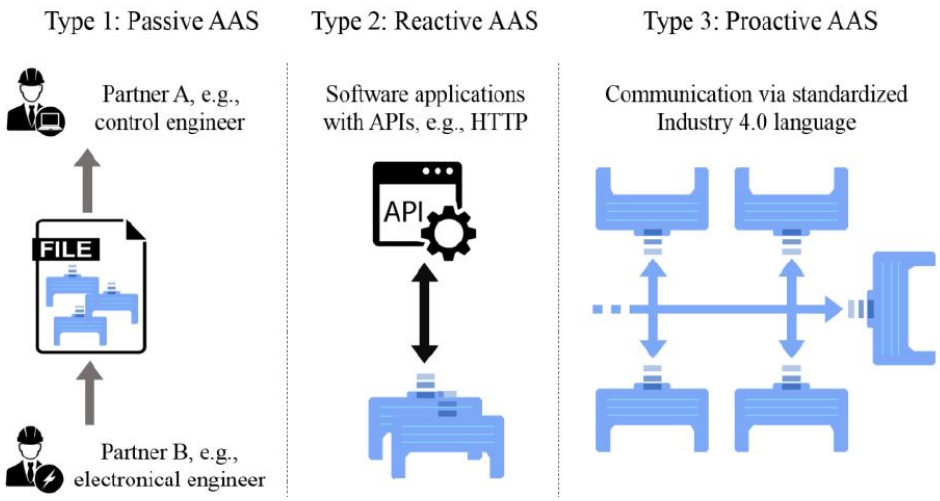


Figure 1.4: Three types of asset administration shell interaction pattern [10]

AAS tools

There exist different open-source tools which provide an implementation of AAS by creating, editing and viewing AAS information in the AASX format:

- Eclipse BaSyx provides the requirements for creating AAS, and enables the integration of existing machines into an Industry 4.0 environment [11].
- PyI40AAS models AAS as python objects, and stores them in CoachDB for easy expansion. It also allows reading and writing of AASX package files [12].
- SAP AAS Service is a docker image based system, which implements the RAMI 4.0 reference architecture including AAS [13].
- The NOVA Asset Administration Shell (NOVAAS) is also a docker image based system developed by NOVA School of Science and Technology [14].
- ConfigWizard is a tool to generate AASs partially automatic. It reduces the development efforts by a graphical user interface [15].
- OpenASS provides a basic sheme on how to implement AAS. However, there still exist some gaps, which prevent the integration of the openAAS Software within a production environment [16].
- AAS-based-PnP-Demo is a Plug-and-Produce based demo system for AAS. It allows to demonstrate the AAS for flexible manufacturing. In other words, this tool applies a PnP system in I4.0 [17, 18].

- AASX Package Explorer (AASX PE) allows not only the experimentation, but also the demonstration of the potentials of the AAS.
  This tool is used in this work, because it has advantages over other tools:
  - It's easy to start to deal with AAS by exchanging and creating files.
  - It creates a visual representation of the contents of the AAS which were stored in a file.
  - It has editing capabilities
  - It has a Graphical user interface.
  - It's convenient for tech-savvy and less technically-inclined users [19].

2 Related works

In this chapter, first a work that simulates an I4.0 scenario is presented, which forms the basis for this work. Next, some papers that represent AAS in different use cases are discussed.

2.1 Model railway demonstrator system

Model railway demonstrator system (MoRa) is an example of Industry 4.0 to define the wireless communication involving the assets required for manufacturing of the product in an industry. MoRa was simulated in [20] using OMNeT++.
As is shown in Figure 2.1 the order comes from the customer terminal, the required raw material or semi-manufactured product for that order will be transfered from material station to the corresponding machine which has the required functionalities to produce the product. Then the finished product goes to the distribution station to get delievered to the customer.
We used this example as a starting point for our project and extended it by adding the negotiation between the production and wireless system.

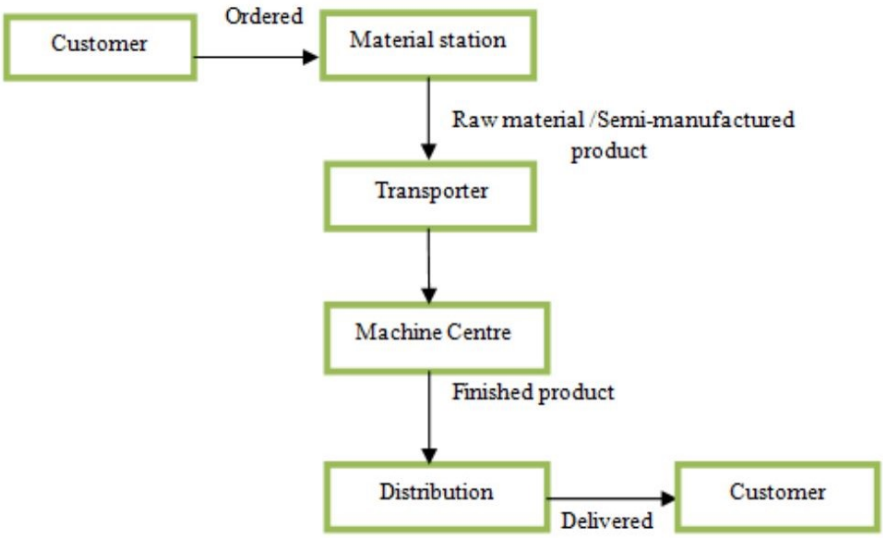


Figure 2.1: Process flow of MoRa [20]

2.2 Other related works on AAS

In this section, we discuss five articles which represent AAS in different aspects using different use cases.
The work [21], focuses on the cloud layers, Open Operator Cloud Platform (OOC) and Common Cloud Central (CCC), and examines the AAS infrastructure extended over these layers. Figure 2.2 shows the AAS related elements of the Open Industry 4.0 (OI4) reference architecture. Here, CCC holds information about assets, provided by Open manufacturers cloud platform to improve the operators digital twin in the OI4 solution framework. It only works as a template repository not an AAS which manages the physical assets. On the other hand, OOC instantiates the AAS by means of an AAS server during the onboarding process of the asset. It receives information about the asset from Open edge computing, and in case of lack of information it can access the CCC. These communications between the cloud layers are based on the concept of the AAS. The implementation of AAS into OI4 is still in progress, and BaSyx will be used.

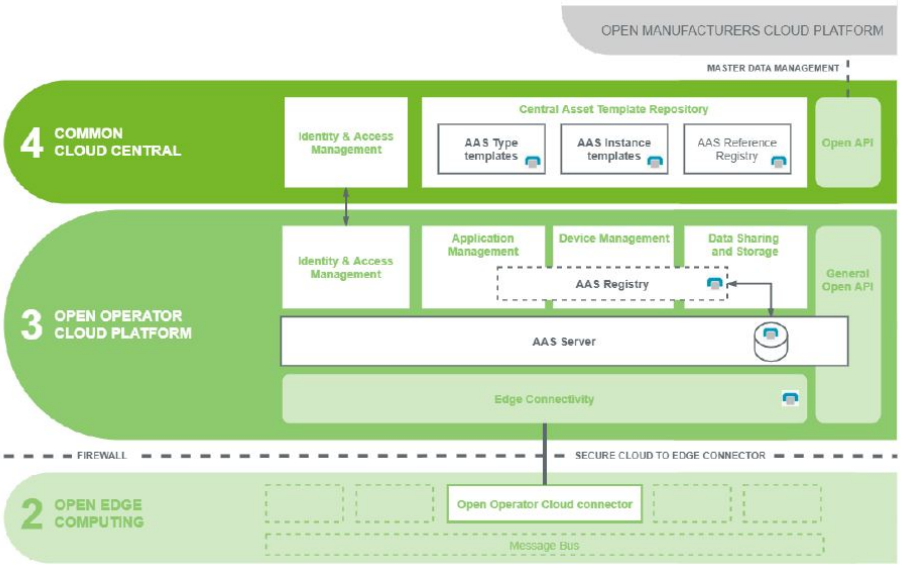


Figure 2.2: AAS related elements of the OI4 reference architecture [21]

The work of [22] implemented AAS for a use case of interoperability, the demonstration manufacturing system. In this example, different communication protocols are used, such as Ethernet Powerlink and Wi-Fi. Here, an AAS gateway is developed to interconnect these heterogeneous protocols via OPC Unified Architecture (OPC-UA). This enables the interaction between different types of assets.
In [23], two AASs are implemented for a robotic arm and a grinding machine to enable interoperability between heterogeneous industrial assets in a manufacturing plant the same as in [22]. Here, there exists also an integrator which is connected to the AASs via OPC-UA. It receives the updates on each asset and visualizes their data for plant managers to know the current state.

The work presented in [17] implements AAS for an industrial digitization scenario with the capability of plug and produce (PnP) of the industrial field devices. This article proposes the AAS-based-PnP-Demo method as mentioned in chapter 1, section 1.2, AAS tools. This scenario contains three robots and a turntable. Each asset has an I4.0 adaptor (an OPC-UA server) to connect to it's AAS. In this work, AutomationML and OPC-UA technologies are used respectively for managing the AAS information, and connecting the AASs. As the result, each robot can be operated independently, and the AAS was demonstrated via a Web User Interface (Web UI).
Since there exists no generally applicable solution for efficient management of the product lifecycle, the work of [24] uses AAS to integrate product lifecycle management (PLM) and application lifecycle management (ALM). This work is using Aasx Package Explorer as a tool to demonstrate the AAS. An AAS for the product is implemented which has two submodels, one to represent the PLM and the other one to represent the ALM data of the product. There is also a RelationshipElement class implemented to work as a link between these submodels.
The work of [25] creates an AAS for an active digital object memory model (ADOMe) for the product within the production. With this approach the product will be no longer a passive object, but an active smart object which is able to collect, interpret and send information. This obviously leads us to a decentralized production process.
To conclude this literature research on articles related to AAS, we can say that, although the works of [22] and [17] indicate that a communication submodel is necessary when modeling the AAS, they do not provide detailed information on that. Moreover, these works do not have the same point of view as this work that consists of modeling the wireless system manager as a communication submodel. This submodel is not only to allow communication between assets, but also to use this information to manage and adapt the production system and the communication resources.
It is also impostant to mention that in this work, we simulated the communication between AASs of different elements of a production system and its communication system. Each of these elements has some properties and functionalities. But, since we are not using any communication technology e.g. OPC-UA to connect this simulation to its real world counterpart as it was done in the related works, this work is only a prove of the concept, and basis for the future work.

3 Model description

This chapter contains the main work of this thesis. Here, the structure of use case scenario is not only presented with state diagrams, but also using an AAS tool called AASX PE, which is discussed in 1.4.

3.1 Wireless industrial automation

A wireless industrial automation system as shown in Figure 3.1, is composed of three systems: physical production system, distributed automation system, and WCS. Each of these possesses multiple elements, respectively called electro-mechanical elements, automation devices, and wireless communication modules or infrastructure devices. The physical system is controlled by the automation system, which relies on the wireless communication system to exchange data among its elements [26]. For instance, in an on/off valve control system, the controller will be notified of a required movement by receiving a message from the production system as an input. The automation system exchanges these data among its elements through the WCS and finally will send a request as an output to the actuator to change its position. To consider the effects of the production system requirements and the conditions of the industrial environment on WCS, this system has been analyzed and modeled for the digital domain.

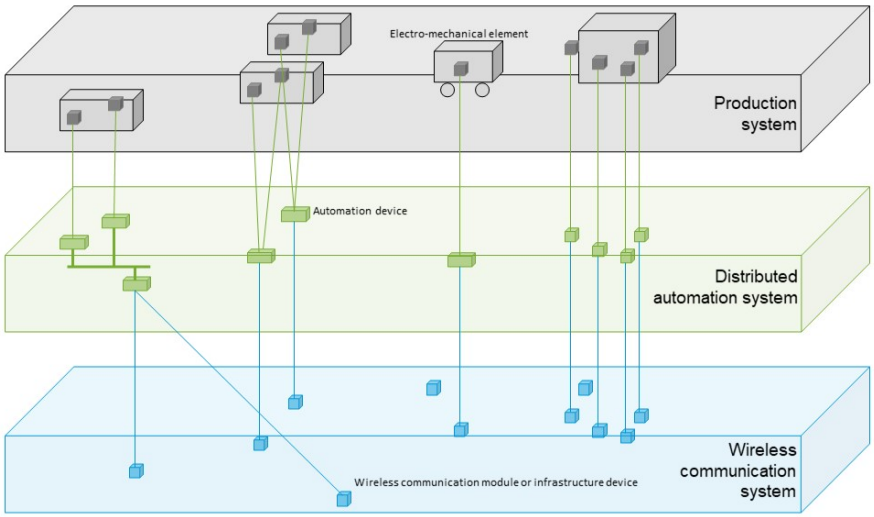


Figure 3.1: Wireless industrial automation as a system of systems [27]

In the concept of AAS, the asset will be transformed from a physical world into a digital

world. These two domains are shown in Figure 3.2. The digital domain (purple layer) is the digital representation of the physical asset developed by AAS. Both layers contain the three layers of Figure 3.1 (i.e., production, automation and communication system). The data which will be exchanged among physical assets is called productive data. The data which will be exchanged among AASs is called I4.0 data. In order for the digital part to be able to monitor and act on the physical part, the domains are connected to each other. For that, management data exchange is necessary. Hence, three types of data exchange are considered in this work:

- Productive data exchange
- I4.0 data exchange
- Management data exchange

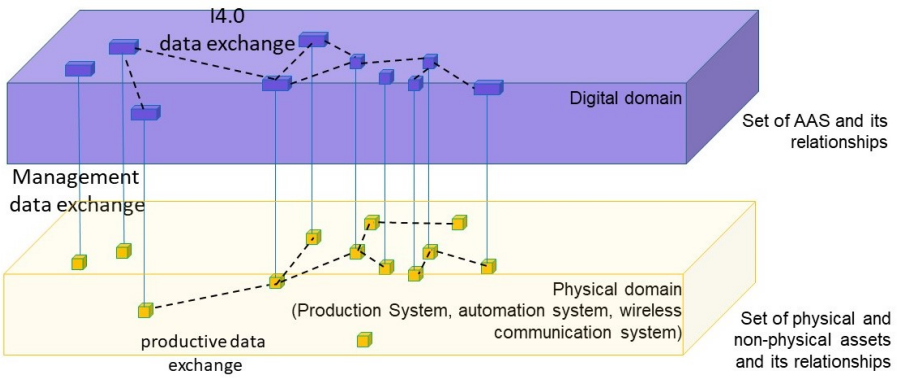


Figure 3.2: Visualization of the physical and digital domain in the concept of I4.0

With this approach, the wireless communication system can be adapted based on production system requirements. The digital part can monitor the production requirements and manage the wireless assets to allocate resources more efficiently. The AAS of the production system can negotiate with the AAS of the wireless system concerning the communication resources. It is necessary to develop AAS for all parties involved to allow this type of negotiation between the wireless system and the automation system. However, due to the high complexity of the wireless system and possibly the automation system, it is important to determine the necessary components of the system, that should be described using AAS. Otherwise, the description of all the existing components in the network may require high and possibly unnecessary efforts. It means that not all physical domain elements have a virtual representation. This is shown in Figure 3.2, where one element in the physical domain does not have a virtual representation, and consequently, is not connected to the digital domain.

3.2 Industry 4.0 scenario for managing communication resources

As mentioned in the last section, the focus of this work is the digital domain (purple layer in Figure 3.2). Using OMNeT++, the concept of collaboration between the production system and WCS is developed. For that, we present an I4.0 scenario as a means to prove the concept. In this scenario, the production system negotiates with the WCS during the production process. Each asset of the system is modeled as an AAS. It means that each module in the simulation is representing its AAS. This model will simulate the I4.0 data exchange between AASs. It means that the virtual representation of the assets can communicate with each other.
Figure 3.3 shows the system classes of this scenario. The modules are customer terminal, product, machine broker, 4 machines with different capabilities and, wireless system manager (WSM) which manages the communication resources.

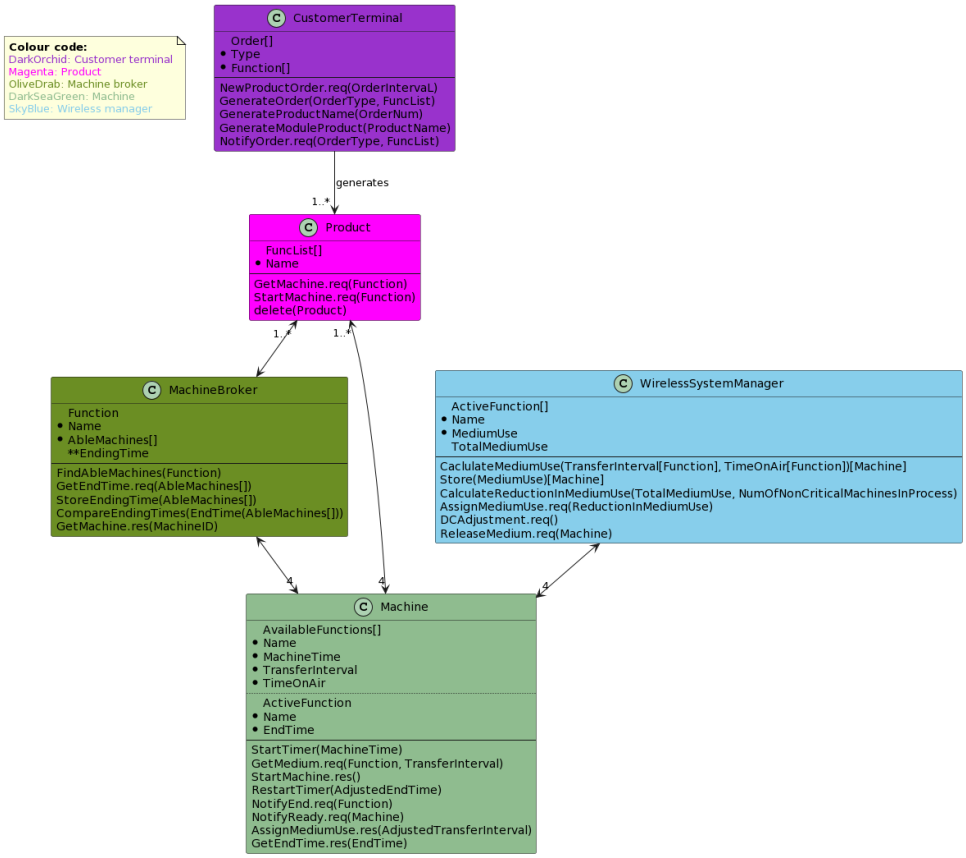


Figure 3.3: System classes of the use case scenario

The scenario is as follows:

1. There are different types of products. Every n second a new order with random (customized or individual) product type is created by the customer terminal. For each order, a digital version of the product is generated by the customer terminal with a product name and the required functions.

2. This digital module is able to interact with the digital versions of other elements. Table 3.1 shows the possible product types with their required functions which is used in this work. Each product interacts with machine broker asking for a machine that can produce the specific function.

Product type	**Required functions**
P1	F1
P2	F2, F1
P3	F3, F2, F4, F1
P4	F3
P5	F4, F3, F2
P6	F3, F4

Table 3.1: The product types and the corresponding required functions to produce them.

3. The machine broker has the information about the machines, their available functions (capabilities) and asks for their ending times. According to these information machine broker assigns a proper machine to the product. Table 3.2 shows the capable machines to operate each function.

Function	**Capable machines**
F1	M0, M1, M2
F2	M0, M1
F3	M0, M1, M3
F4	M3

Table 3.2: The functions and the corresponding capable machines to operate them.

4. Each machine has all the information about the functions. Table 3.3 shows the transfer intervals of the functions. The machine time of each function will be calculated by multiplying its transfer interval with the number of required messages to operate the processing function. We assume this to be 1000 messages for all of the functions. The assigned machine sends a request for communication resources to the WSM after it starts to operate.

Function	**Transfer interval(ms)**
F1	0.5
F2	0.3
F3	0.2
F4	0.7

Table 3.3: The functions and the corresponding transfer intervals.

5. As a result, the WSM will have access to the asset-related data continuously and will be able to keep the use of the wireless medium at an efficient level. This part is the concentration of the work which proves the concept of negotiation between the production system and the WCS. The sequence diagram in Figure 3.9 shows this negotiation in details.

3.3 Detailed behavior of each AAS

In this section, the detailed behavior of the system classes shown in Figure 3.3 are visualized as state diagrams using Unified Modeling Language (UML). Furthermore, each diagram is explained in words for a better understanding of the reader.
Before we start to describe the diagrams, it is required to explain some basic definitions:

Finite State Machine (FSM) is a behavior model, which consists of a finite number of states. There exists always an initial state, which starts the machine by transitioning to other states whenever it receives a specific input. Each state generates outputs by reacting on the inputs [28].

State diagram is a graphical visualization of the state machine. This diagram helps to represent the entire life-cycle of objects [29].

Please refer to the color code shown in figure 3.4 for all of the diagrams.

Customer terminal

This module is the initializer of the simulation. According to Figure 3.4, it has been programmed in two states. The state "Init" is only to show the initialization. It means, whenever the first self message (NewProductOrder.ind) arrives, the state transition occurs. In state "Idle", the customer terminal first generates a random order type (GenerateOrder), then creates a name for the product of that type (CreateProductName) using the order number, which starts with one for the first order, and increases one by one for the next ones. After that, the module product will be generated (GenerateModuleProduct). Customer terminal will notify the module Product of the order type, and its required functions list (NotidyOrder.req) as shown in table 3.1.
The self message to create a new order arrives every 10 seconds. In other words, every 10 seconds, a random order type, and its corresponding product module will be generated.

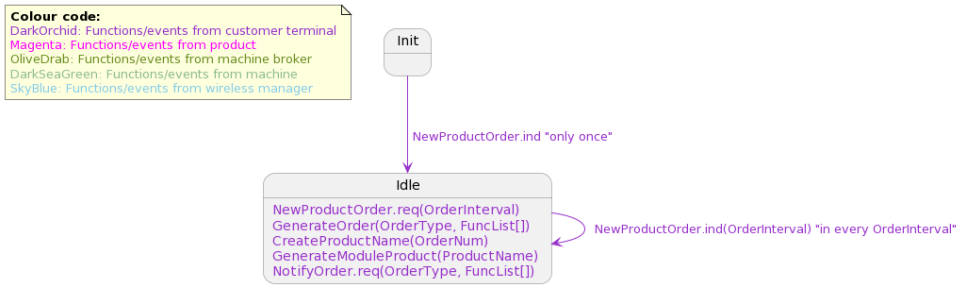


Figure 3.4: State diagram of the customer terminal

Product

This module is a dynamic module created by customer terminal for each order as explained above. Its state diagram is illustrated in Figure 3.5. By receiving the notification of a new order (NotifyOrder.ind) from customer terminal, this module sends a request (GetMachine.req) to the machine broker, which is explained below. By this request the product asks for an available machine to operate its first required function. Whenever it receives the machine ID of the available machine (GetMachine.cnf) from machine broker, it checks if the machine is available at the current simulation time.

- If yes, the state transits to "Assigned to machine". In this state the product sends a request to the corresponding machine to operate the function (StartMachine.req).
- If no, the state transits to "Wait". In this state the product schedules a self message (StartMachine.req) by the ending time of the machine. With this message product will know when to ask the machine to start the operation. Whenever the self message arrives, the state transits to "Assigned to machine", and sends the request to the machine.

The next state transition occurs by receiving the confirmation from the machine (StartMachine.cnf), notifying that it started the operation on the product. The new state is called "In progress". In this state the product waits for the machine to finish the operation. By receiving the indication that the process has been ended (NotifyEnd.ind), the product checks if the processed function was the last function.

- If yes, the product is ready to deliver to the customer. Therefore, it is programmed for the module product to delete itself at this point of the simulation.
- If no, it transits to state "Function ready" to request the machine broker for the next machine (GetMachine.req). After receiving the response, the decision making occurs again to see if the machine is available immediately or not.

These last two parts show the decision making capability of the product independently from other management and decision making units. This can be seen as a step towards a decentralized manufacturing process.

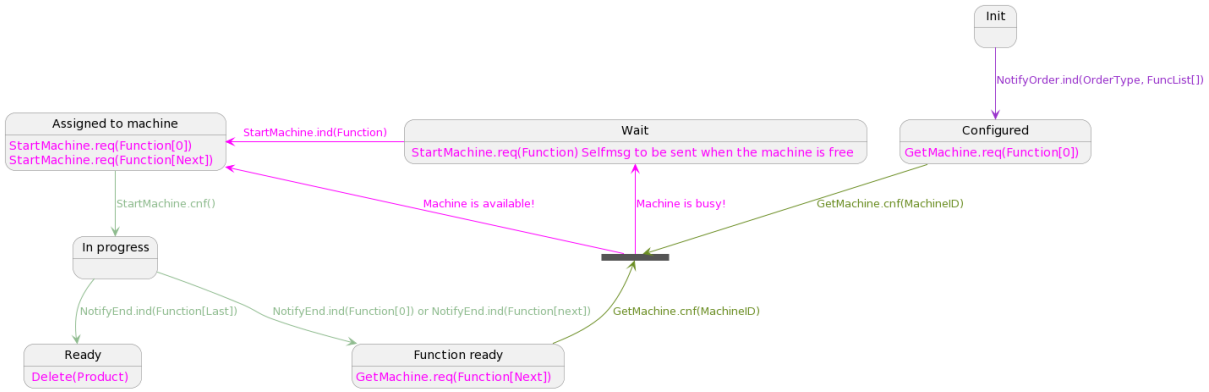


Figure 3.5: State diagram of the product

Machine broker

The state diagram of Machine broker is shown in Figure 3.6. This module works as a decision making unit for the plant. It searches for the first available machine to operate the function requested from the product (GetMachine.ind). First, it finds all machines which are capable to operate the function (FindAbleMachines), then it asks all of them for the ending time (GetEndTime.req). By receiving each response from the machines (Endtime.cnf), it stores the ending times in a vector (StoreEndingTime). Whenever the response from the last capable machine arrives, it starts to compare the ending times to choose the first available machine (CompareEndingTimes). As the result, it informs the product about the chosen machine by sending a response (GetMachine.res).

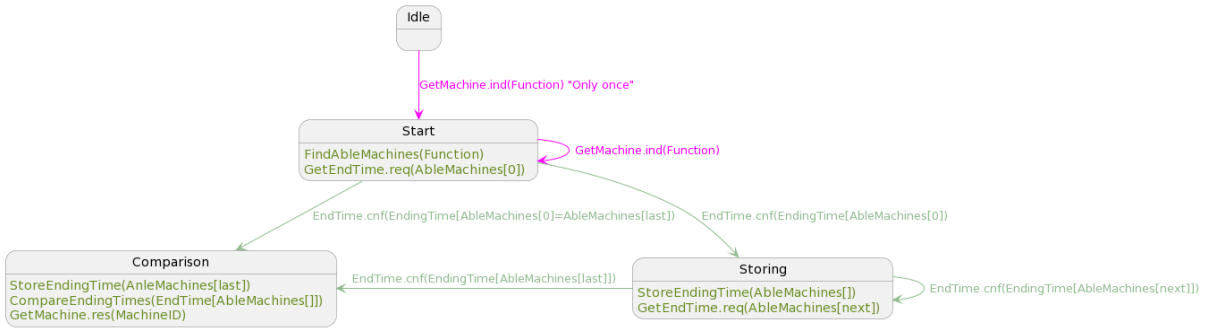


Figure 3.6: State diagram of the machine broker

Machine

The Figure 3.7 shows that the module machine is programmed in three states. When it is in "Idle" it means that the machine is free. It might receive the request for ending time (GetEndTime.ind) either in this state or in the state "In progress". Because state "Started" is only a transient state.

By receiving the request to start the machine (StartMachine.ind) from the product, the state transits to "Started". The machine starts the timer (StartTimer), and informs the product (StartMachine.res) and the wireless system manager (GetMedium.req) by sending

messages and the state transits to "In progress". As soon as Wireless system manager knows that the machine started an operation, it will calculate the machine's medium use using the transfer interval of the function in progress. This is why the request from the machine contains the transfer interval.
Whenever the medium use exceeds the defined limitations (in this case 50 percent), the wireless manager sends a request to the machine to reconfigure its medium use by slowing down the operation (AssignMediumUse.ind). The machine restarts the timer (Restart-Timer) with the delayed machine time (See Section 3.4 for definitions and calculations), and informs the wireless manager by sending a response (AssignMediumUse.res).

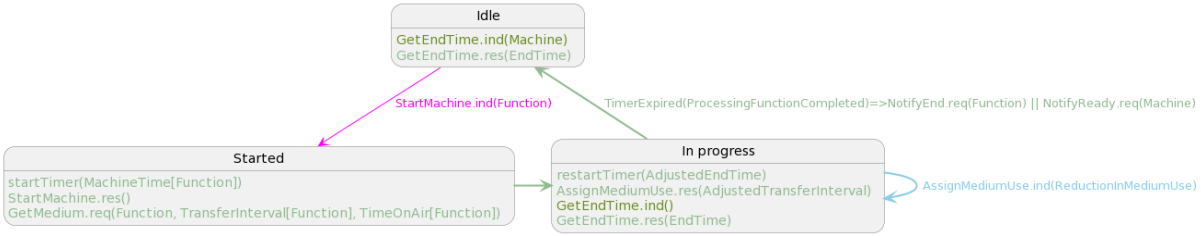


Figure 3.7: State diagram of the machine

Wireless system manager

As mentioned before, this module is the concentration of this work. As is shown in Figure 3.8, it starts in the state "Idle" and transits to three different states depending on the messages it receives. Each of these states are explained below.

- CalculateMediumUse
  By receiving the request to calculate the medium use (GetMedium.ind), which informs the WSM that the machine started an operation, the finite state machine goes to this state. Here, the WSM calculates the medium use (CalculateMediumUse) using the transfer interval and time on air (See Section 3.4), which is a constant value of 0.1 in this example. Then it stores the value in a vector (Store).
  WSM checks this vector continously. Whenever the sum of the medium uses of all of the machines exceeds 50 percent, it sends a self message called (DCAdjustent.req) to request for configuration. There exists also another condition which can lead to this state, which is explained below.

- AssignMediumUse
  By receiving the self message (DCAdjustment.req), WSM warns itself about the required configuration in the amount of medium used by the machines. In this state, the amount of required reduction in medium use for each machine will be calculated (CalculateReductionInMediumUse), and sent to the corresponding machines (AssignMEdiumUse.req). The machines will adjust their use of medium by

slowing down the operation. As soon as the WSM receives the confirmation from the machines (AssignMediumUse.cnf), it will go back to the state "CalculateMediumUse" to calculate and store the adjusted medium use.
Here, we consider one function (Function 4) to be a critical function, which does not have to slow down the operation in any case. For this reason, any machine which is processing function 4 will not receive any request for configuration. Therefore, the total amount of reduction needed in medium use will be divided between all other machines. This will create some errors which will be discussed with its possible solutions in Chapter 5.

- ReleaseMediumUse
  Whenever a machine has finished an operation, it will notify the WSM (NotifyReady.ind) to change the machine's medium use back to zero (Store(MediumUsee=0)). This is what happens in this state.

Figure 3.9 visualizes the scenario if we connect these AASs to the real world. It shows how the AAS of the machine helps the communication between the machine and WSM by analysing, and passing the information between these two in real time. In other words, it analyses the situation and tells the machine what to do. Generally, this is what happens between all of the AASs and their assets.

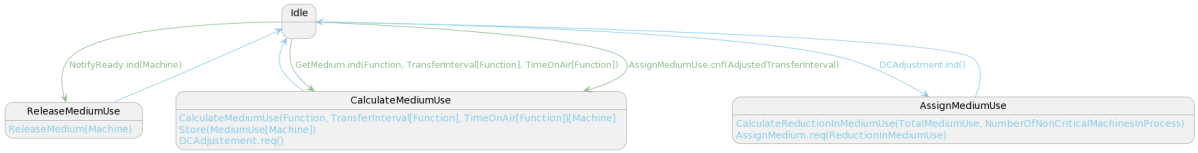


Figure 3.8: State diagram of the wireless system manager

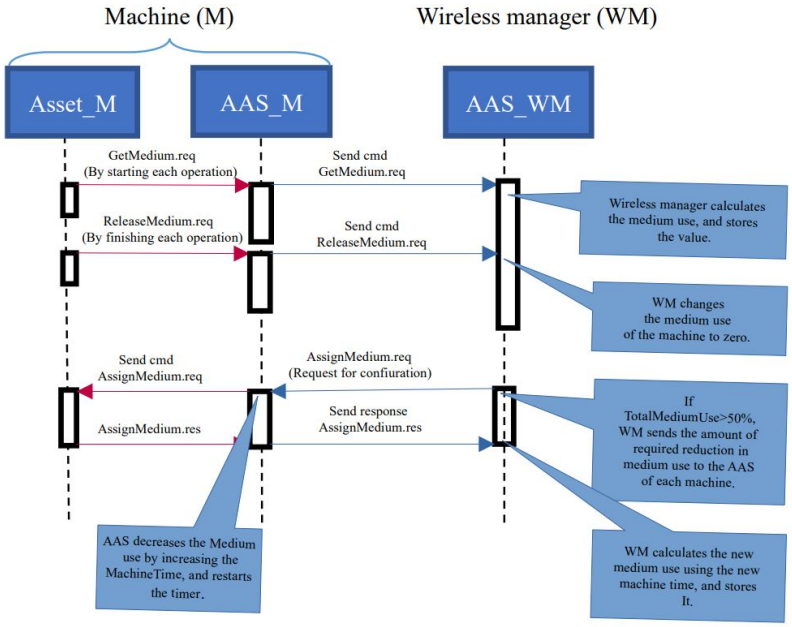


Figure 3.9: The scenario for the wireless manager and the use of AAS

3.4 Definitions and mathematical calculations

Machine time (MT) is the time that a machine needs to operate a function. This value differs for each function. It will be calculated by multiplying the transfer interval with the number of required messages to operate a function (NumberOfMsges). We assume this value to be 1000 for all of the functions.

$$MT = TI \times NumberOfMsges \tag{3.1}$$

Transfer interval (TI) is the time interval between each message to be sent. This value is specific to each function. See Table 3.3.

Time on air (TOA) is a duration in which a message is on its way to the receiver. This is assumed to be 0.1 for all of the functions.

Medium use is a percentage of wireless medium that a machine uses during an operation. This will be calculated by dividing time on air to the transfer interval.

$$MediumUse = \frac{TOA}{TI} \times 100 \tag{3.2}$$

Reduction in medium use is the amount of medium use which has to be reduced by a machine. Here, we set a maximum limit for the medium use which is 0.5 (50%). Then, whenever it exceeds this limit, the medium use should be adjusted back to 0.5. This value will be calculated by subtracting 0.5 from the total medium use of

the system. Then, we have to divide this amount to all of the machines, which are processing non-critical functions (F1, F2 or F3).

$$ReductionInMediumUse = \frac{TotalMediumUse - 0.5}{NumberOfNonCriticalMachinesInProcess} \quad (3.3)$$

At some point in the simulation, this calculation creates a problems, which will be discussed in Chapter 5.

Adjusted transfer interval is the new transfer interval after the machine has slowed down the operation.

$$AjustedTI = \frac{TOA}{MediumUse - ReductionInMediumUse} \quad (3.4)$$

Adjusted machine time is the new machine time after the machine has slowed down the operation.

$$AdjustedMT = AdjustedTI \times NumberOfMsges \quad (3.5)$$

Adjusted end time is the new ending time after the machine has slowed down the operation.

$$AdjustedET = ET + (AdjustedMT - MT) \quad (3.6)$$

3.5 The structure of each AAS

In this section, the structure of each AAS is shown using Aasx PE discussed in section 1.4. For that, a template submodel for the state machine is used.
Before we start to describe the structure, it is required to explain some keywords used in AAS [30]:

Submodel (SM) is a model included in the AAS which refers to a well-defined domain or subject matter. Submodels are technically separated from each other. There exist two types of submodels: Template and instance.

Submodel element is an element to describe and differentiate the assets. Operations can be modeled using a submodel element.

SubmodelElementCollection (SMC) is a collection of submodel elements. SMC can be a reference to a submodel element.

Property (Prop) defines the characteristic to describe and differentiate the products or components. There exist two types of property: Type and instance.

MultiLanguageProperty (MLP) is a property that has a value that can be provided in multiple languages.

Operation/Method is an executable realization of a function with a name and a list of parameters.

In Figure 3.10 it can be seen that each module is described as a submodel for the standardized I4.0 language. Each SM consists of several SMCs, and a PNG file of the



Figure 3.10: I4.0 language in AASX PE

module's state diagram. The common SMCs of all modules are:

- Constants: The constants which are used in the module are described here.
- Variables: The variables which are used in the module are described here.
- States: The states of the state machine of the module are listed here.
- Transitions: Here the transitions from one state to another state are described. The inputs and outputs of the transitions are also added as operations.

Additionally, for each module, a state-transition table is represented to demonstrate the details about each transition legibly.

Customer terminal

Table 3.4, and Figure 3.11 show that whenever the self message **"NewProductOrder.ind"** arrives at the customer terminal while it is in the state **Init**, the state will change to **Idle**. Consequently, some actions which are programmed to be executed at this state such as **GeneraeOrder**, and **GenerateModuleProduct** which are shown in Figure 3.4 will be called. As a result of these actions, the message **"NotifyOrder.req"** will be sent out to the product. This message will act as an output for this transition and as an input for the transition from state **Init** to state **Configured** in the module product (See Table 3.5).
If the customer terminal receives the self message while it is in the state **Idle**, a self-transition will happen. Here, the state does not change, and the actions of this state will be executed again. This event is shown with an arrow pointing back to the state **Idle** in Figure 3.4.
Each input and output might carry some parameters. In this case the message **"Notify-Order.req"** is carrying the variables **OrderType** and **FuncList** to the other module.

Input	Input parameters	Current state	Next state	Output	Output parameters
NewProductOrder .ind	-	Init	Idle	NotifyOrder.req	OrderType, FuncList
NewProductOrder .ind	-	Idle	Idle	NotifyOrder.req	OrderType, FuncList

Table 3.4: The state-transition table of customer terminal

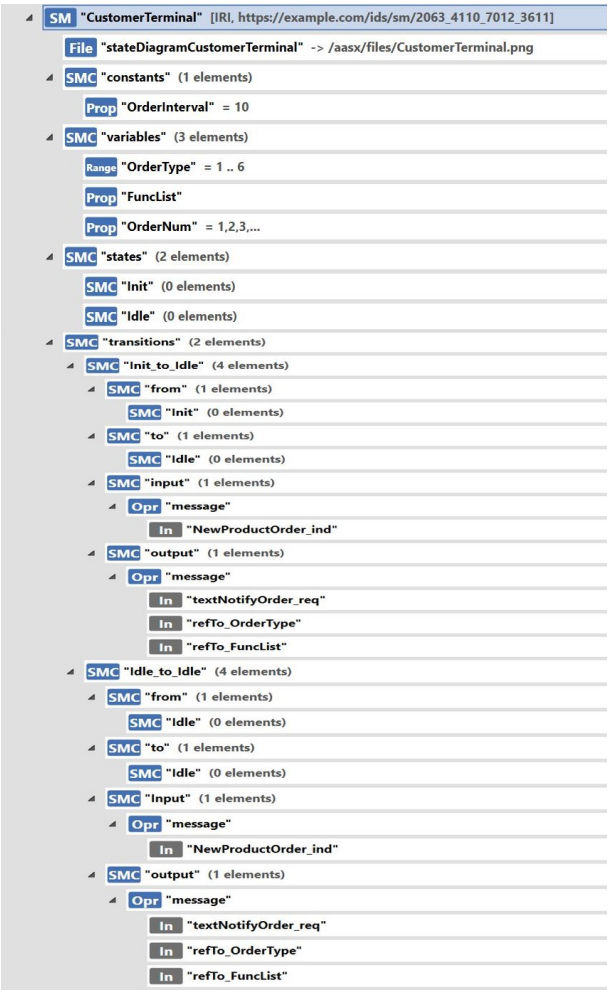


Figure 3.11: Structure of customer terminal in AASX PE

Product

As shown in Figure 3.12 an extra SMC called "nameplate" has been defined for the module product to identify the product with information such as the name of the product, order type, the list of required functions, and the production date. Here, it is possible to add more descriptive information, e.g order date, if needed.

It is important to mention that to show the decision-making abilities of the product, boolean variables such as **MachineIsAvailable** and **LastFunction** are used in the structure

of the state machine. When the product receives the message **"GetMachine.cnf"** in the state **Configured** it will check if the machine is available or not. If yes, the variable **MachineIsAvailable** will be equal to one, and the state will change to **Assigned to machine**. If no, the variable will be equal to zero, and the state will change to **Wait**. The same thing happens with the variable **LastFunction** in the state **In progress**. If it is the last function in the **FuncList** and the product is ready to deliver to the customer, then the variable will be equal to one, and the product goes to state **Ready** to delete the module from the simulation. If it is not the last function the variable will be equal to zero and the product goes to state **Function ready** to ask for the next machine.
It is also important to mention that as is shown in Table 3.5, not all transitions result in an output that triggers another transition in the same module or other modules. Sometimes the actions which are programmed to be executed in a state do not have any output.

Input	**Input parameters**	**Current state**	**Next state**	**Output**	**Output parameters**
NotifyOrder.ind	OrderType, FuncList	Init	Configured	GetMachine.req	Function
GetMachine.cnf and MachineIsAvailable =0	MachineID	Configured	Wait	StartMachine.req	Function
GetMachine.cnf and MachineIsAvailable =1	MachineID	Configured	Assigned to machine	StartMachine.req	Function
StartMachine.ind	Function	Wait	Assigned to machine	StartMachine.req	Function
StartMachine.cnf	-	Assigned to machine	In progress	-	-
NotifyEnd.ind and LastFunction=0	Function	In progress	Function ready	GetMachine.req	Function
GetMachine.cnf and MachineIsAvailable =0	MachineID	Function ready	Wait	StartMachine.req	Function
GetMachine.cnf and MachineIsAvailable =1	MachineID	Function ready	Assigned to machine	StartMachine.req	Function
NotifyEnd.ind and LastFunction=1	Function	In progress	Ready	-	-

Table 3.5: The state-transition table of product



Figure 3.12: Structure of product in AASX PE

Machine broker

A boolean variable is also used to show the behavior of the machine broker in a structured form. When the machine broker receives the **EndingTime** of a machine in state **Start**, it will check if the machine is the last machine in the **AbleMachinesList** or not. If not, the state changes to **Storing** to store the value and send the request to the next machine in the list. If yes, the state changes to **Comparison** to store the value and send the **"GetMachine.res"** to the product. There is no need to compare since there exists only one capable machine to operate the function. The same thing happens in the state **Storing**.

Input	Input parameters	Current state	Next state	Output	Output parameters
GetMachine.ind	Function	Idle	Start	GetEndTime.req	MachineID
GetMachine.ind	Function	Start	Start	GetEndTime.req	MachineID
EndTime.cnf and LastMachine=0	EndingTime	Start	Storing	GetEndTime.req	MachineID
EndTime.cnf and LastMachine=1	EndingTime	Start	Comparison	GetMachine.res	MachineID
EndTime.cnf and LastMachine=0	EndingTime	Storing	Storing	GetEndTime.req	MachineID
EndTime.cnf and LastMachine=1	EndingTime	Storing	Comparison	GetMachine.res	MachineID

Table 3.6: The state-transition table of machine broker



Figure 3.13: Structure of machine broker in AASX PE

Machine

A template submodel called "Nameplate" provided by Platform I4.0 [31] is used to describe the machine. This submodel is to keep the identifying and descriptive information of an industrial equipment. Some of this information are as follows:

MachineType (Range) is the type of machine which can be 0, 1, 2, or 3 in our use case. This information was not contained in the Nameplate. But, it is a value similar to SerialNumber.

ManufacturerName (MLP) is a legally valid name of the natural or judicial person which is directly responsible for the design, production, packaging, and labeling of the machine and bringing it into circulation.

ManufacturerProductDesignation (MLP) is a short description of the machine.

SerialNumber (Prop) is a unique combination of numbers and letters to identify the machine.

YearOfConstruction (Prop) is the year that the machine has been manufactured.

It should be mentioned that here we face an exceptional situation in the transition from state **Started** to state **In progress**. This transition does not require any input. It happens automatically whenever the action of state **Started** is executed completely. The same thing happens with some transitions in the module wireless system manager (See Table 3.8).
As is shown in Table 3.7 sometimes we have multiple inputs or outputs. There might be a case where both inputs are required for the transition, and as the result, we will have both outputs. This is indicated with an **and** between the two inputs and outputs. As opposed to this, **or** shows a situation where only one of the inputs is enough for the transition.

Input	Input parameters	Current state	Next state	Output	Output parameters
StartMachine.ind	Function	Idle	Started	StartMachine.res and GetMedium.req	Function, Transfer-Interval
-	-	Started	In progress	-	-
AssignMediumUse-.ind or GetEndTime.ind	Reduction-In-MediumUse	In progress	In progress	AssignMediumUse-.res or GetEndTime.res	Adjusted-Transfer-Interval, or EndingTime
ProcessingFunction-Completed	-	In Progress	Idle	NotifyEnd.req and NotifyReady.req	Function, and MachineID

Table 3.7: The state-transition table of machine



Figure 3.14: Structure of machine in AASX PE

Wireless system manager

As is shown in Table 3.8 there exists an exceptional case in the transition from state **Idle** to state **CalculateMediumUse**. This transition will only have an output if a condition is true. It means, only if the total medium use exceeds the limitation which is defined for it, the self message will be sent to adjust the medium use of the system.

Input	Input parameters	Current state	Next state	Output	Output parameters
GetMedium.ind or AssignMediumUse-.cnf	Function, Transfer-Interval or Adjusted-Transfer-interval	Idle	Calculate-Medium-Use	If TotalMediumUse >50% then DCAdjustment.req	-
-	-	Calculate-Medium-Use	Idle	-	-
NotifyReady.ind	MachineID	Idle	Release-Medium-Use	-	-
-	-	Release-Medium-Use	Idle	-	-
DCAdjustment.ind	-	Idle	Assign-Medium-Use	AssignMediumUse-.req	Reduction-In-MediumUse
-	-	Assign-Medium-Use	Idle	-	-

Table 3.8: The state-transition table of wireless system manager



Figure 3.15: Structure of wireless system manager in AASX PE

4 Communication models in SIMBA

Second part of the thesis is explained in this chapter. First, a motivation is given. Then, the approaches to reach the objective are explained.

4.1 Motivation

SIMBA is a simulation system for innovative model-based solution concepts for manifold applications in the water, wastewater and biogas areas developed by IFAK [32]. This software is not developed to handle communication base models. However, in this chapter we made it possible by implementing a simple peer-to-peer communication system called TicToc in SIMBA. Moreover, we present the next necessary steps for implementing the negotiation scenario, used in this work, as an example of a more complicated communication system.

4.2 TicToc example in SIMBA

TicToc is an example used as a tutorial for OMNeT++ [33]. It is a network that consists of two nodes. One of the nodes will create a packet, and the two nodes will keep passing the same packet back and forth. The nodes are called Tic and Toc.
Since the existing blocks in SIMBA are not capable of sending messages, it is necessary to extend a default class called "siModule"(see siModuleTempl.cs). to implement this model. This class represents a default block with all the methods that can be overwritten to define new blocks with different behaviors. In this work, a new class which inherits from "siModule" is created and all the methods which are necessary for TicToc example are overwritten to describe the block's behavior. In this class, we have different regions. The ones which are useful for the tictoc model are:

- DefaultInterface member
  - InitParameter() is a method to define the block's parameters. This method makes it possible for users to enter the value of the parameters by clicking on the block in SIMBA. Within this method, The parameters which are needed to represent to the users will be defined. This window is shown in Figure 4.1.
  - UpdateParameter() is a method to read and use the parameters. If the user enters the parameters and clicks on okay, then UpdateParameter will be called.

  - DefineIOs() is a method to define the inputs and outputs of a block.

- DiscreteSys member
  - getX0d() is a discrete state vector.
  - xdfun() is the update function of the discrete state vector.



Figure 4.1: Parameter block window in SIMBA

This section represents seven steps to implement the TicToc example in SIMBA:

1. Create a new c project in visual studio.
   Add these references from SIMBA installation to the project: sisicore.dll, cslapack.dll, wUtil.dll, wconsoleA.dll, xyplotcontrol.dll, wsvg.dll, wEditor.dll, exprComp.dll. For example, sisicore.dll should be there for our project to know about the siModule.

2. Create an Omnet base message passing block for SIMBA by creating a class named "omnetModuleBase" which inherits from "siModule" (see framework.cs). Only overwrite the parts which are necessary for the model.
   - Add the following class to the block:
     - queue-state: This class inherits from an existing SIMBA class called dynamicState (sisicore.dll). For each output, we have a message queue to keep the messages of that output, as long as they are not sent to the destination block, and are on the way. In other words, the list of channels are organized in this class.
   - Add the following functions to the block:
     - Xmlsave(): This function is to save the state.
     - Xmlread(): This function is to allow to read the state back.
       These functions make it possible to save the state of the simulation at any time into a file, and run the file later at that state, and continue the simulation from that state.
     - ReadRes() and Writers(): SIMBA stores all the internal signals and states for every simulated point in time to enlarge the result file. This state is stored in a binary file (Not XML file for performance reasons).

3. Create a .cs file called tictoc1 which contains two classes:
   - A class named "mymessage" derived from "messageObjectBase" to define the send address, destination address and the content of the message.
   - A class called "Tic" which inherits from "omnetModuleBase" to create a default block for tic and toc nodes.
     In this class, two internal member variables should be added:
     - initMsg (Boolean): This is to define if the specific instance of the block initially sends a message or not (initializes the message passing).
     - MaxMessage (double): It stops message passing after x messages received.

     Now, the functions should be described:
     - initialize(): This is to start the simulation.
     - handleMessage(): This is to specify the behavior of the block (what to do after receiving the message). Here it is simply sending the same message out.
     - InitParameter(): Here add the initMsg and maxMessages to the parameter list, so the user can specify them by clicking on the block. We can also create a finite state machine (In our example it chooses randomly between input and output to send the message, since we have bi-directional channels connecting the blocks).
     - UpdateParameter(): This is to read out from the parameters list to see which values they have. Here we also initialize for each output a channel. We are also able to create bi-directional channel here.
     - defineIOs(): Define inputs and outputs of the block.
     - statevector(): Use it to count the number of messages passed.

   In this part, you can create as many blocks as you need. If you have different modules with different behaviors and characteristics, then you might need one class for each of them.

4. Compile the code. Then the file omnetS.dll will be copied to the folder example in the project, and this is the pre-condition for SIMBA to know about the new block.

5. Open the project "tictoc.simu" in the folder example in SIMBA.

6. Choose any block of SIMBA, and then right-click on the block, choose select block class. Choose the omnetS.dll to see all the SIMBA blocks defined in that .dll file. (All classes derived from "siModule"). This window is shown in Figure 4.2.

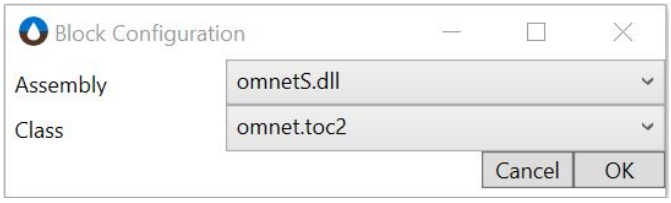


Figure 4.2: Block configuration window in SIMBA

7. Run the simulation and see the results (excel file).
   Figure 4.3 shows the graphic result in SIMBA. The channel is bi-directional, and the message passes between two nodes back and forth.

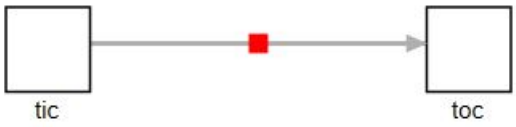


Figure 4.3: TicToc example in SIMBA

4.3 Negotiation scenario in SIMBA

Using the capabilities we reached by developing the TicToc example, a basic model for negotiation scenario is created as is shown in Figure 4.4. In this implementation, the modules only pass messages randomly between eachother. The specific behaviors of each module are still to be implemented.

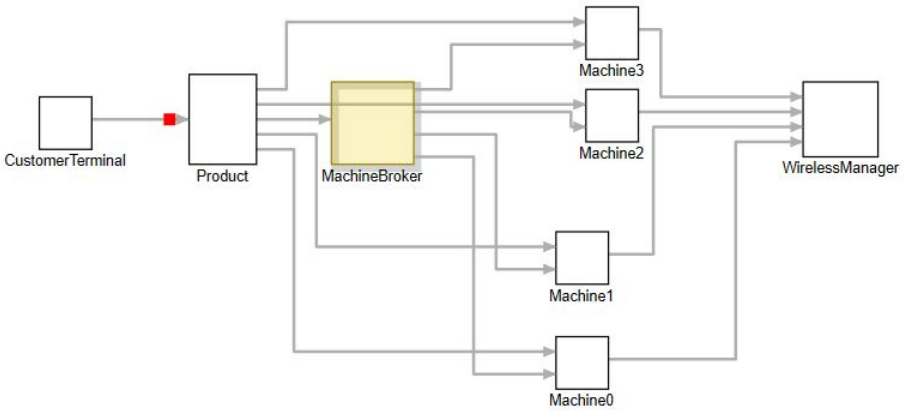


Figure 4.4: Use case scenario in SIMBA

The next steps are as follows:

- Find a way to implement TCP and UDP client and server apps. Socket programming might be needed, or If we could use inet libraries in SIMBA this might be also a solution.

- Create base omnet blocks for CustomerTerminal, MachineBroker, WirelessManager, and MachineTools. It is possible to define more than one input and output if needed in defineIOs() function, and define the different behaviors in handleMessage() function or using FSM.
- Create a dynamic module for the product, which will be generated every time we have a new order and will be deleted whenever the order is ready. Each product module has a unique id and can communicate with other modules the same as a base omnet block.
- Create a message type that is able to carry several variables (the same as .msg files in omnet++)

5 Results and Discussion

Figure 5.1 is the graphical view of the simulation in OMNeT++. All of the modules explained in chapter 3 are present in this result. Moreover, there exists a fifo (First In First Out) queue. This queue makes the parallel product handling possible, in the way that it helps the machine broker to work on the requests one by one.

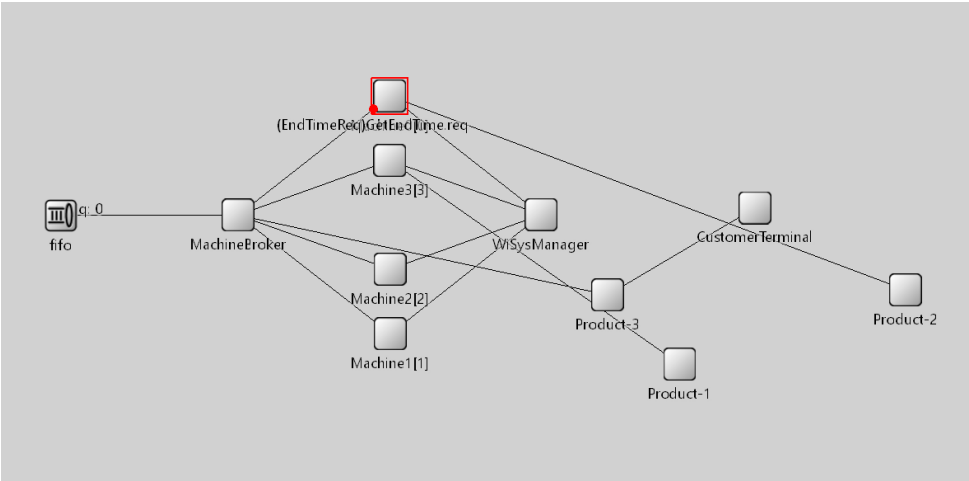


Figure 5.1: Use case scenario in OMNeT++

As the result of this work, the simulation can only handle 4 products through the scenario due to some restrictions in the wireless system manager. In this section, these problems and the possible solutions will be presented for the future work. The restrictions are as follows:

1. If we only have the critical function in process, and at the same time the medium use exceeds the limit. This situation will cause a problem since WCS does not send any request for configuration to the machine.
   - What is implemented:
     It will only print “error“, if this situation happens.
   - First possible solution:
     We can prevent this situation completely, if whenever a product is assigned to a machine, the machine checks the total medium use, and if there is not enough resources, it rejects the request.
     This solution has contrast with the concept of smart manufacturing. Because the products will not be able to get produced in parallel, or at least in an efficient time. They have to wait until one operation is done and then, start

to get produced. But, if the operation could get started, and then, the WSM adjusts the medium use, in this case, every machine could be working only in a slower speed. This scenario would be closer to what we try to reach in I4.0. Therefore, we suggest a second solution.

- Second possible solution:
  We assume the products have priority to each other. We set a parameter named as "level of importance" for each product. This helps the WSM to decide which machines to slow down.
  In case of having only the critical function in process, the WSM checks the priority of the products and sends the request for configuration to those with lower priority.

2. After the reduction needed in medium use is calculated, this amount can be less than the amount of medium that a machine is using. This might cause a problem, since the machine will have a negative medium use after the adjustment.

   - What is implemented:
     Whenever this happens, those machines with less medium use than the required amount of reduction won't receive any request to configuration. WSM calculates the reduction needed in medium use for the machines which are left. Again, it will check if the medium use of the remaining machines is more than the amount of required reduction. If yes, it will send the request. If no, it will print "error".
     This is not a good solution. If we only filter the machines once, we still might get error the next time we calculate the required amount of reduction. And if we filter the machines in a loop, we might run out of machines which we are allowed to ask for configuration.

   - Possible solution:
     WSM should be able to calculate the amount of reduction in medium use in a different way. It should consider the ratio of each machine's medium use to the total medium use, and ask the machine to reduce the medium use in the same ratio. In this way, we will not get into the situation where the medium use of the machine is smaller than the required amount of reduction.
     The amount of reduction needed in medium use for Machine n (n=0,1,2,3) can be calculated using this equation:

$$ReductionInMediumUse_{Mn} = \frac{MediumUse_{Mn}}{TotalMediumUse} \times (TotalMediumUse - 0.5) \quad (5.1)$$

6 Conclusions and Future work

In this work, the collaboration between production and wireless communication system is enabled. With this approach, WCS can reach the level of adaptability required by I4.0 in a way that it monitors, controls and adapts the communication resources to keep them at an efficient level.

The presented results are the fundament for the future work. First, the restrictions discussed in chapter 5 should be solved for the simulation to get closer to a real case scenario with a high number of production. After that, this digital domain can get connected to a simulated physical domain by integrating the AAS into the simulation. This can be done by implementing the UDP communication to allow OPC UA data exchange. Subsequently, it will be possible to simulate productive and management data exchange.

As a result of the second part of the thesis, the base block to implement communication models in SIMBA is available. This block, and the roadmap provided in this work are the start point for the future work.

Bibliography

[1] Eric Howard. *The Evolution of the Industrial Ages: Industry 1.0 to 4.0*, September 2018.

[2] Trend Micro Research. *Security in the Era of Industry 4.0: Dealing With Threats to Smart Manufacturing Environments*, April 2019.

[3] I. Sadaf Khan; M. Ovais Ahmad; J. Majava. *Industry 4.0 and sustainable development: A systematic mapping of triple bottom line, Circular Economy and Sustainable Business Models perspectives.* Journal of cleaner production, July 2020.

[4] `https://www.tutorialspoint.com/wireless_communication/wireless_communication_overview.htm`.

[5] `https://www.asmltd.com/introduction-wireless-telecommunication/`.

[6] B. Boss; S. Malakuti; S. Lin; Thomas Usländer; E. Clauer; M. Hoffmeister; L. Stojanovic. *Digital Twin and Asset Administration Shell Concepts and Application in the Industrial Internet and Industrie 4.0.* Industrial Internet Consortium &Plattform Industrie 4.0, September 2020.

[7] Stefan Pollmeier (ESR Pollmeier GmbH) Dr. Jörg Neidig (Siemens AG), Andreas Orzelski (Phoenix Contact GmbH Co. KG). Asset administration shell reading guide. January 2022.

[8] *Industrial-process measurement, control and automation – Smart Manufacturing – Part 1: Terms and definitions*, May 2021.

[9] German Electrical and Electronic Manufacturers' Association. *Examples of the Asset Administration Shell for Industrie 4.0 Components, Basic Part, White paper*, April 2017.

[10] Xun Ye; Seung Ho Hong; Won Seok Song; Yu Chul Kim; Xiongfeng Zhang. An industry 4.0 asset administration shell enabled digital solution for robot-based manufacturing systems. October 2021.

[11] `https://projects.eclipse.org/projects/technology.basyx`.

[12] `https://git.rwth-aachen.de/acplt/pyi40aas`.

[13] https://github.com/SAP/i40-aas.

[14] https://gitlab.com/novaas/catalog/nova-school-of-science-and-technology/novaas.

[15] T.; Marcon P.; Bradac Z.; Schröder T.; Belyaev A.; Werner T.; Braun V.; Kamensky P.; Zezulka F.; et al. Arm, J.; Benesl. *Automated Design and Integration of Asset Administration Shells in Components of Industry 4.0*, March 2021.

[16] http://acplt.github.io/openAAS/.

[17] Xun Ye, Junhui Jiang, Changdae Lee, Namhyeok Kim, Mengmeng Yu, and Seung Ho Hong. Toward the plug-and-produce capability for industry 4.0: An asset administration shell approach. *IEEE Industrial Electronics Magazine*, 14(4):146–157, 2020.

[18] https://github.com/hanyangyexun/AAS-based-PnP-Demo.

[19] https://github.com/admin-shell-io/aasx-server.

[20] Grushika Malve. *Simulation Platform for Wireless Industry 4.0 Communication Scenarios*, December 2019. .

[21] *The Asset Administration Shell in the OI4 solution framework, White Paper Version 1.0*, July 2021.

[22] Xun Ye and Seung Ho Hong. Toward industry 4.0 components: Insights into and implementation of asset administration shells. *IEEE Industrial Electronics Magazine*, 13(1):13–25, 2019.

[23] Miguel A. Iñigo, Alain Porto, Blanca Kremer, Alain Perez, Felix Larrinaga, and Javier Cuenca. Towards an asset administration shell scenario: a use case for interoperability and standardization in industry 4.0. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, pages 1–6, 2020.

[24] Andreas Deuter and Sebastian Imort. Plm/alm integration with the asset administration shell. *Procedia Manufacturing*, 52:234–240, 01 2020.

[25] Max Birtel; William Motsch; Jens Popper; Martin Ruskowski. Method for the development of an asset administration shell in a product-driven modular production – realizing an active digital object memory. 2020.

[26] Performance testing of 5g systems for industrial automation. *5G Alliance for Connected Industries and Automation white paper*.

[27] L. Underberg and L. Rauchhaupt. Performance testing of novel wireless communication networks for industrial automation on the example of 5g. *Conference on Communication in Automation (KommA)*, 2019.

[28] `https://www.itemis.com/en/yakindu/state-machine/documentation/user-guide/overview_what_are_state_machines`.

[29] `https://www.microtool.de/en/knowledge-base/what-is-a-state-diagram/`.

[30] Details of the asset administration shell, part 1 - the exchange of information between partners in the value chain of industrie 4.0 (version 3.0rc01). November 2020.

[31] Plattform Industrie 4.0. Submodel templates of the asset administration shell. November 2020.

[32] Trend Micro Research. `https://www.ifak.eu/en/products/simba`.

[33] Ahmet Sekercioglu. *OMNeT++ Tutorials and Technical Articles*. Monash University. `https://docs.omnetpp.org/tutorials/tictoc/`.